

DIY Challenge Robot

Nav2 Tuning Guide

MPPI Controller & 8 Critics • Costmap Parameters
Open Outdoor vs Tight Indoor Tuning • Failure Mode Diagnosis
ROS 2 Humble • nav2_params.yaml reference values included

Table of Contents

Section	Topic
1	Nav2 Stack Overview for This Robot
2	MPPI Controller — How It Works
3	The 8 MPPI Critics — Current Weights & Effects
4	Costmap Parameters Explained
5	Planner Parameters (SmacPlannerHybrid)
6	Scenario Tuning: Open Outdoor vs Tight Indoor
7	Common Nav2 Failure Modes & Parameter Fixes
8	Behavior Server & Recovery Tuning
9	Quick-Reference Parameter Table

1 Nav2 Stack Overview for This Robot

This robot uses the Nav2 Humble stack with MPPI as the local trajectory controller and SmacPlannerHybrid as the global path planner. The main configuration file is `src/challenge_bringup/config/nav2_params.yaml`.

Component	Plugin / Node	Frequency	Purpose
BT Navigator	bt_navigator	on goal	Behaviour-tree goal execution
Global Planner	SmacPlannerHybrid	10 Hz	A* Hybrid-A* (kinematic) path
Local Controller	MPPIController	20 Hz	Sampled trajectory optimization
Global Costmap	2-D costmap	1 Hz	Inflation + global obstacles
Local Costmap	2-D + Voxellayer	10 Hz	Rolling 4×4m window
Behavior Server	bt_navigator	on request	Spin, backup, wait recoveries
Collision Monitor	collision_monitor	10 Hz	Hard stop on imminent collision

1.1 Robot Footprint & Velocity Limits

From `nav2_params.yaml`:

```
# Robot footprint (metres) – outer corners of 406mm square chassis
footprint: "[[0.35, 0.20], [0.35, -0.20], [-0.35, -0.20], [-0.35, 0.20]]"
```

```
# MPPI velocity envelope
FollowPath:
  vx_max: 0.6    # m/s forward max
  vx_min: -0.1   # m/s reverse max
  vy_max: 0.0    # differential drive – no lateral
  wz_max: 1.2    # rad/s yaw rate max
  controller_frequency: 20.0
```

2 MPPI Controller — How It Works

Model Predictive Path Integral (MPPI) is a sampling-based controller. At each 20 Hz control cycle it simulates **1,000 random trajectories** forward by 40 time steps of 0.05 s each (= 2 seconds look-ahead). Each trajectory is scored by a weighted sum of critic costs. The optimal control is the weighted average of all trajectories, biased toward the lowest-cost ones.

2.1 Key MPPI Parameters

nav2_params.yaml — FollowPath:

MPPIController:

```
time_steps: 40          # prediction horizon (steps)
model_dt: 0.05          # seconds per step → 2 s look-ahead
batch_size: 1000        # trajectory samples per cycle
vx_std: 0.2             # velocity noise sigma (forward)
wz_std: 0.4             # angular velocity noise sigma
temperature: 0.3        # softmax sharpness (lower = sharper selection)
gamma: 0.015            # discount factor (lower = more greedy)
iteration_count: 1      # optimisation iterations per cycle
```

2.2 Tuning Philosophy

- **batch_size**: Higher = better coverage of trajectory space but more CPU. 1000 is the practical limit for Jetson at 20 Hz.
- **time_steps × model_dt**: The planning horizon. 2 s at 0.6 m/s = 1.2 m look-ahead. Increase time_steps for faster courses; decrease for tight manoeuvres.
- **temperature**: Lower values make the controller more aggressive in selecting low-cost trajectories; higher values average more trajectories (smoother but slower).

NOTE: MPPI runs entirely on CPU in Nav2 Humble — all 1000 trajectories are evaluated sequentially on the ARM cores. There is no GPU acceleration in this stack.

3 The 8 MPPI Critics — Current Weights & Effects

Each critic scores every sampled trajectory. Critics with higher weight dominate the final trajectory selection. The current values below are from `src/challenge_bringup/config/nav2_params.yaml`.

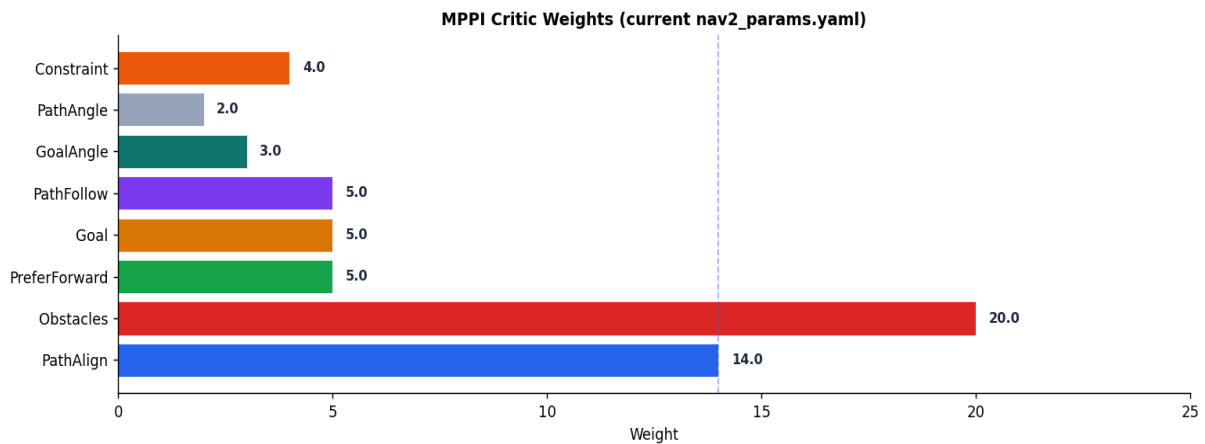


Figure 1 — Current MPPI critic weights. *ObstaclesCritic critical_weight=20 dominates collision avoidance; PathAlignCritic=14 dominates path tracking.*

Critic	Weight	What It Penalises	Tuning Effect
PathAlignCritic	14.0	Path occupancy ratio — penalises trajectories that deviate from the global path	Higher → robot tracks global plan closely; lower → more free-space shortcuts
ObstaclesCritic (critical_weight)	20.0	Proximity to obstacles below critical threshold (0.10 m); collision_cost=10000	Always keep high; collision_cost makes crossing threshold catastrophic
ObstaclesCritic (repulsion_weight)	1.5	Repulsion cost in inflation zone beyond collision threshold	Higher → robot avoids inflation zone more aggressively
GoalCritic	5.0 (threshold 1.4 m)	Distance to goal within threshold_to_consider radius	Active only near goal; higher → faster final approach
PreferForwardCritic	5.0	Backward motion — penalises negative vx	Higher → robot avoids reversing; lower → more willing to back up
PathFollowCritic	5.0 (offset 5)	Distance from trajectory point to corresponding path point at offset_from_furthest	Higher → strong position tracking; lower → only direction tracking
GoalAngleCritic	3.0 (threshold 0.5 m)	Heading error at goal within threshold_to_consider	Higher → robot arrives more accurately oriented; active in final 0.5 m
PathAngleCritic	2.0	Angle between trajectory direction and path direction at offset_from_furthest=20	Higher → robot aligns heading with path more strictly during travel
ConstraintCritic	4.0	Hard velocity constraints violation — penalises trajectories outside vx/wz bounds	Should stay at 4.0+; going too low risks constraint violations in the trajectory

4 Costmap Parameters Explained

4.1 Global Costmap

nav2_params.yaml — global_costmap:

```
global_costmap:
  global_costmap:
    ros__parameters:
      update_frequency: 1.0
      publish_frequency: 1.0
      global_frame: map
      robot_base_frame: base_link
      use_sim_time: false
      robot_radius: 0.35          # inscribed radius of footprint
      resolution: 0.05           # metres per cell — 5 cm
      track_unknown_space: true
      plugins: ["static_layer", "obstacle_layer", "inflation_layer"]

    inflation_layer:
      cost_scaling_factor: 3.0    # exponential decay — higher = faster falloff
      inflation_radius: 0.45      # metres around obstacles to inflate

    obstacle_layer:
      raytrace_max_range: 6.0     # max range for ray-clearing free space
      obstacle_max_range: 5.0     # max range for marking obstacles
```

4.2 Local Costmap (Rolling Window)

nav2_params.yaml — local_costmap:

```
local_costmap:
  local_costmap:
    ros__parameters:
      update_frequency: 10.0
      publish_frequency: 10.0
      global_frame: odom
      robot_base_frame: base_link
      rolling_window: true
      width: 4                   # metres — 4×4 m window
      height: 4
      resolution: 0.05           # 5 cm cells
      plugins: ["voxel_layer", "inflation_layer"]

    voxel_layer:
      z_resolution: 0.05         # vertical cell size
      z_voxels: 16               # 16 × 0.05 = 0.8 m height

    inflation_layer:
      cost_scaling_factor: 3.0
      inflation_radius: 0.40      # slightly tighter than global
```

4.3 Inflation Radius Tuning

- **inflation_radius** should be at least your robot half-width + safety margin. Current: 0.40–0.45 m for a 0.35 m half-width robot — 5–10 cm margin.
- **cost_scaling_factor**: higher values mean cost falls off faster with distance. 3.0 creates a medium-soft gradient. Increase to 5–8 for open courses; keep at 3 for tight.

- Global costmap inflation can be wider (0.45) than local (0.40) because the global planner benefits from earlier obstacle avoidance at path-planning time.

5 Planner Parameters (SmacPlannerHybrid)

SmacPlannerHybrid implements Hybrid-A* — a state-lattice planner that respects non-holonomic differential drive kinematics. It produces smooth, kinematically feasible global paths.

nav2_params.yaml — SmacPlannerHybrid:

GridBased:

```
plugin: "nav2_smac_planner/SmacPlannerHybrid"
downsample_costmap: false
downsampling_factor: 1
tolerance: 0.25           # metres – goal tolerance for planner
allow_unknown: true       # plan through unknown cells
max_iterations: 1000000
max_on_approach_iterations: 1000
max_planning_time: 5.0    # seconds budget per plan
motion_model_for_search: "REEDS_SHEPP"
angle_quantization_bins: 72
analytic_expansion_ratio: 3.5
minimum_turning_radius: 0.40 # metres – derived from wz_max / vx_max
reverse_penalty: 2.1
change_penalty: 0.0
non_straight_penalty: 1.20
cost_penalty: 2.0
retrospective_penalty: 0.025
lookup_table_size: 20.0
cache_obstacle_heuristic: false
debug_visualizations: false
expected_planner_frequency: 10.0
```

5.1 Key Planner Tuning Points

- **minimum_turning_radius:** Keep at $\geq$ the physical turning radius. Computed as: $\text{min_turning_radius} \approx \text{vx_max} / \text{wz_max} = 0.6 / 1.2 = 0.5$ m. Current value 0.40 m is slightly aggressive — increase to 0.5 if plans look jagged.
- **reverse_penalty:** $2.1 \times$ cost for reversing. Increase to 5.0+ if robot reverses too aggressively.
- **non_straight_penalty:** Extra cost for turning maneuvers. Increase to 1.5–2.0 if planner produces unnecessarily sinuous paths.
- **allow_unknown=true:** Lets the planner route through unexplored regions. Set false if the full map is known to prevent phantom routing.

6 Scenario Tuning: Open Outdoor vs Tight Indoor

6.1 Open Outdoor Course (DIY Challenge Typical)

The competition course is a large outdoor area with sparse obstacles (cones, barriers, natural terrain). Priorities: speed, path tracking, minimal unnecessary turns.

Parameter	Current	Recommended Range	Rationale
vx_max	0.6	0.5–0.8	Rules allow up to ~1 m/s; 0.6 is conservative
PathAlignCritic weight	14.0	12–18	High weight to stay on global plan in open space
ObstaclesCritic repulsion_weight	1.5	1.0–2.0	Soft repulsion OK; no narrow corridors
inflation_radius (local)	0.40	0.35–0.45	Full robot width + small margin
time_steps	40	40–60	More look-ahead helps at higher speed
minimum_turning_radius (planner)	0.40	0.5–0.7	Match actual turn capability at speed
cost_scaling_factor	3.0	2.0–4.0	Softer gradient OK with sparse obstacles

6.2 Tight Indoor / Narrow Corridor Course

If the course includes corridors, doorways, or narrow gates, the robot must navigate more carefully at lower speed.

Parameter	Suggested Change	Rationale
vx_max	0.3–0.4 m/s	Slower allows more time to respond to obstacles
PathAlignCritic weight	8–10	Reduce to allow slight detours around tight obstacles
ObstaclesCritic repulsion_weight	3.0–5.0	Strong repulsion from walls critical in corridors
inflation_radius (local)	0.25–0.30	Tighter inflation to fit through narrow gaps
cost_scaling_factor	5.0–8.0	Sharp gradient forces robot to stay central in corridors
time_steps	25–30	Shorter horizon for slow-speed precision manoeuvring
minimum_turning_radius	0.25–0.35	Allow tighter turns for door-clearing

7 Common Nav2 Failure Modes & Parameter Fixes

Symptom: Robot oscillates around path	Cause: Too-high PathAlignCritic or PathFollowCritic weight	Fix: Reduce PathAlignCritic from 14 → 10; increase temperature from 0.3 → 0.5 for smoother selection
Symptom: Robot cuts corners dangerously	Cause: PathAlignCritic weight too low OR inflation_radius too small	Fix: Increase PathAlignCritic weight; increase inflation_radius by 0.05 m increments
Symptom: Robot freezes near obstacle (no path)	Cause: Inflation too wide; costmap too conservative	Fix: Reduce local inflation_radius; check global costmap for phantom obstacles (stale cells)
Symptom: Robot never reaches goal (tolerance failure)	Cause: xy_goal_tolerance too tight; goal in costmap obstacle	Fix: Set xy_goal_tolerance: 0.30; verify goal is not inside an inflated obstacle cell
Symptom: Robot reverses unexpectedly during navigation	Cause: reverse_penalty too low; path requires reverse	Fix: Increase reverse_penalty to 3.0–5.0 in SmacPlannerHybrid; add PreferForwardCritic weight
Symptom: Robot spins in place endlessly	Cause: Recovery loop triggered; costmap stuck obstacle	Fix: Clear costmap: ros2 service call /global_costmap/clear_entirely_global_costmap ; reduce recovery max_rotational_vel
Symptom: Slow, jerky motion in open space	Cause: batch_size / time_steps too low for the speed	Fix: Increase time_steps to 56 (= 2.8 s horizon); increase batch_size if CPU allows
Symptom: Goal tolerance never met (yaw)	Cause: yaw_goal_tolerance too tight (0.20 rad ≈ 11°)	Fix: Increase yaw_goal_tolerance to 0.35–0.50 if heading precision not required

8 Behavior Server & Recovery Tuning

The nav2_behaviors server provides recovery actions triggered by the Behaviour Tree when the robot gets stuck. Current configured behaviors: Spin, BackUp, DriveOnHeading, Wait.

nav2_params.yaml — behavior_server:

```
behavior_server:
  ros__parameters:
    costmap_topic: local_costmap/costmap_raw
    footprint_topic: local_costmap/published_footprint
    cycle_frequency: 10.0
    behavior_plugins: ["spin", "backup", "drive_on_heading", "wait"]

    spin:
      plugin: "nav2_behaviors/Spin"
    backup:
      plugin: "nav2_behaviors/BackUp"
    drive_on_heading:
      plugin: "nav2_behaviors/DriveOnHeading"
    wait:
      plugin: "nav2_behaviors/Wait"

    # Speed limits during recoveries
    max_rotational_vel: 1.0      # rad/s (was 3.2 accel limit)
    min_rotational_vel: 0.4
    rotational_acc_lim: 3.2
```

8.1 Recovery Tuning

- **max_rotational_vel:** 1.0 rad/s is the spin recovery rate. Reduce to 0.5 for tighter spaces; increase for faster recovery exit.
- **Spin recovery:** triggered when the robot is stuck and cannot plan. If spin does not clear the blockage, check for real physical obstructions.
- **BackUp recovery:** drives backward 0.3 m by default. The distance and speed are set in the Behaviour Tree XML — edit `navigate_w_replanning_and_recovery.xml` to change.
- **Wait recovery:** waits for a dynamic obstacle to move. Useful if the course has human or robot cross-traffic.
- Recovery sequence (default BT): try plan → spin → backup → replan. After N failures the goal is aborted.

9 Quick-Reference Parameter Table

All values from `src/challenge_bringup/config/nav2_params.yaml`.

Parameter	Value	Location
<code>controller_frequency</code>	20 Hz	<code>controller_server</code>
<code>planner_frequency</code>	10 Hz	<code>planner_server.expected_planner_frequency</code>
<code>vx_max / vx_min / wz_max</code>	0.6 / -0.1 / 1.2	<code>FollowPath.MotionModel</code>
<code>time_steps / model_dt</code>	40 / 0.05 s = 2 s horizon	<code>FollowPath.MPPIController</code>
<code>batch_size</code>	1000	<code>FollowPath.MPPIController</code>
<code>temperature / gamma</code>	0.3 / 0.015	<code>FollowPath.MPPIController</code>
<code>PathAlignCritic weight</code>	14.0	<code>critics[PathAlignCritic]</code>
<code>ObstaclesCritic critical_weight</code>	20.0	<code>critics[ObstaclesCritic]</code>
<code>collision_cost</code>	10000.0	<code>critics[ObstaclesCritic]</code>
<code>collision_margin_distance</code>	0.10 m	<code>critics[ObstaclesCritic]</code>
<code>GoalCritic threshold_to_consider</code>	1.4 m	<code>critics[GoalCritic]</code>
<code>GoalAngleCritic threshold</code>	0.5 m	<code>critics[GoalAngleCritic]</code>
<code>xy_goal_tolerance</code>	0.20 m	<code>goal_checker</code>
<code>yaw_goal_tolerance</code>	0.20 rad (~11°)	<code>goal_checker</code>
<code>required_movement_radius</code>	0.3 m in 8 s	<code>progress_checker</code>
<code>global_inflation_radius</code>	0.45 m	<code>global_costmap.inflation_layer</code>
<code>local_inflation_radius</code>	0.40 m	<code>local_costmap.inflation_layer</code>
<code>cost_scaling_factor</code>	3.0	both costmaps
<code>global_resolution</code>	0.05 m/cell	<code>global_costmap</code>
<code>local_size / resolution</code>	4×4 m / 0.05 m	<code>local_costmap</code>
<code>minimum_turning_radius</code>	0.40 m	<code>SmacPlannerHybrid</code>
<code>reverse_penalty</code>	2.1	<code>SmacPlannerHybrid</code>
<code>max_rotational_vel (recovery)</code>	1.0 rad/s	<code>behavior_server</code>

End of Nav2 Tuning Guide. For the full YAML see `src/challenge_bringup/config/nav2_params.yaml`. For environment setup and launch, see the [Competition Day Playbook](#).